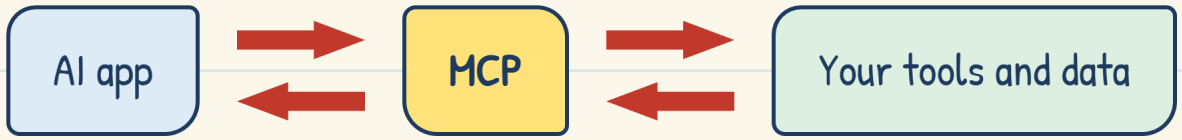
CLASS NOTES · SIMPLE ENGLISH · COMPLETE OVERVIEW

MCP Notes

Model Context Protocol: the complete SOP

What MCP is, how it is built, the JSON-RPC messages it sends, and the steps to connect local and remote servers.



Based on MCP spec 2026-07-28
Notes written October 2026

SOP map: what you will learn

1. **The idea**
   What MCP is, why it exists, RAG versus MCP
2. **Architecture**
   Host, client, server, layers, primitives, flow
3. **JSON-RPC**
   Requests, responses, notifications, errors
4. **Transports**
   stdio, Streamable HTTP, local versus remote
5. **Connect and build**
   Steps for local and remote servers, FastMCP
6. **Wrap-up**
   Connectors, 2026 changes, safety, cheat sheet

Read the parts in order. Each part builds on the one before it.

What is MCP?

MCP = Model Context Protocol

An open standard that lets AI apps connect to tools and data in one common way.

The USB-C idea

One standard port. Any AI app (the laptop) can use any MCP server (the device) that fits the port.

What MCP does

It defines how an AI app asks a server for tools, data and prompt templates, and how the server answers.

What it does not do

It does not choose your AI model or decide how the app uses it. MCP is only the connection.

Why do we need it?

Before MCP

$$4 \times 5 = 20$$

4 AI apps and 5 tools need 20 custom connections. Each one is separate code to build and to fix.

With MCP

$$4 + 5 = 9$$

Each tool gets one MCP server. Each AI app gets one MCP client. They all fit together.

Write a server once. Every MCP client can use it.

Agentic RAG project vs MCP

Question	Agentic RAG (last class)	MCP (this module)
What do you build?	The complete app, like a whole restaurant.	One tool server, like a USB device.
Who is the user?	The end customer who asks questions.	Anyone using an AI app such as Claude Desktop, Claude Code or Cursor.
Who makes the screen?	You write the UI and the logic.	The AI app already has its own UI.
What is the goal?	Answer customer queries from a knowledge base.	Give an existing AI app new abilities.

Your RAG project is not lost. It can become an MCP server, and then Claude Desktop can use it too.

Example: GitHub through MCP

You
"List the open issues in my repository."

→

Claude
The AI app. It holds an MCP client.

→

GitHub MCP server
Offers a tool that lists issues.

→

GitHub
The real service that holds your data.

1. Claude learns what the server can do (**tools/list**) and sees the issues tool.
2. Claude calls that tool (**tools/call**) with your repository name.
3. The server sends back the issues. Claude reads them and answers you.

No new app was built. Companies do the same with Jira, Slack and internal databases.

Why connect a local project?

Before

You open each file in VS Code, edit it by hand, and run it yourself.

After (File System MCP server)

You say what you want in chat. Claude reads and searches your files, creates and edits them, and asks your approval first.

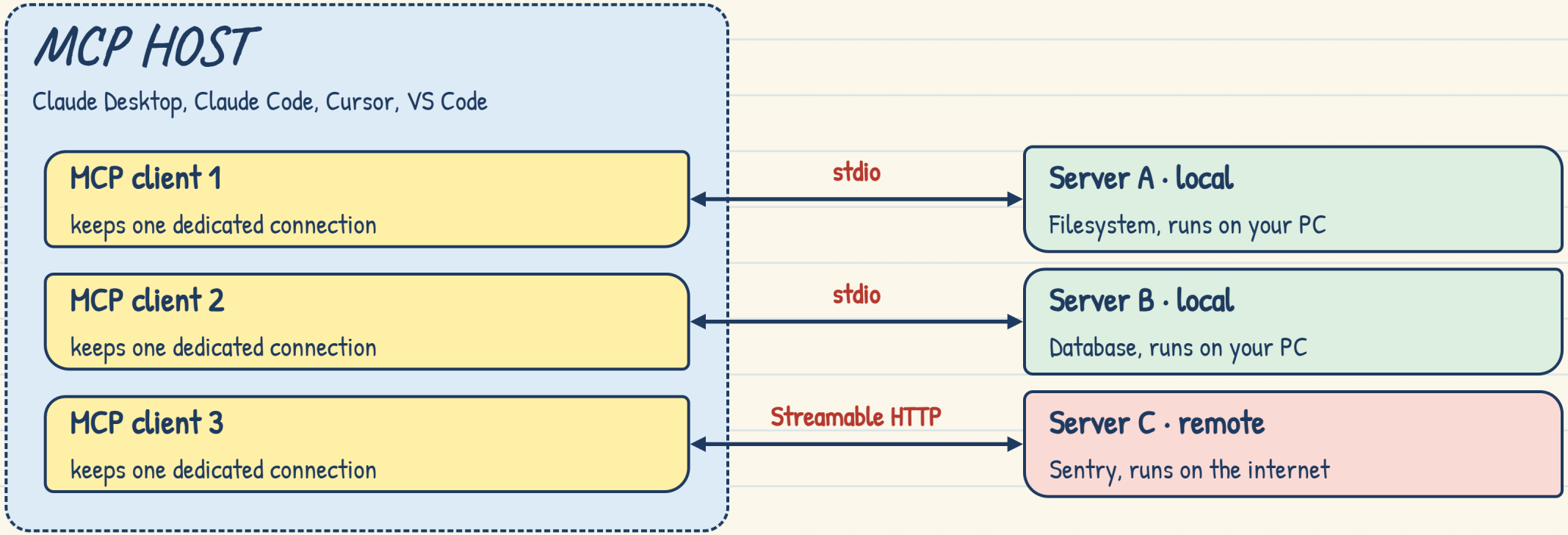
Remember this correction

The File System server reads, searches, creates and edits files. It does **not** run your code. To run code you need another tool, such as a terminal tool.

Stay safe

Share only folders you are happy for Claude to read and change. The server runs with your own user permissions.

Architecture: host, client, server



One host makes one client for each server. A server is the program that serves context, wherever it runs.

Two layers of MCP

Transport layer (outer)

How the bytes travel: stdio or Streamable HTTP. It handles the connection, message framing and sign-in (for HTTP).

Data layer (inner)

- JSON-RPC 2.0 messages
- Discovery: server/discover
- Primitives: tools, resources, prompts
- Notifications and progress updates

Simple rule

Data layer = WHAT we say.

Transport layer = HOW we send it.

The same JSON-RPC message works on every transport.

Three server primitives

Tools

Actions. Functions that do something: file operations, API calls, database queries.

Controlled by: the AI model decides when to call them.

```
tools/list
tools/call
```

Resources

Data. Context to read: file contents, database records, API responses.

Controlled by: the app decides what to include.

```
resources/list
resources/read
```

Prompts

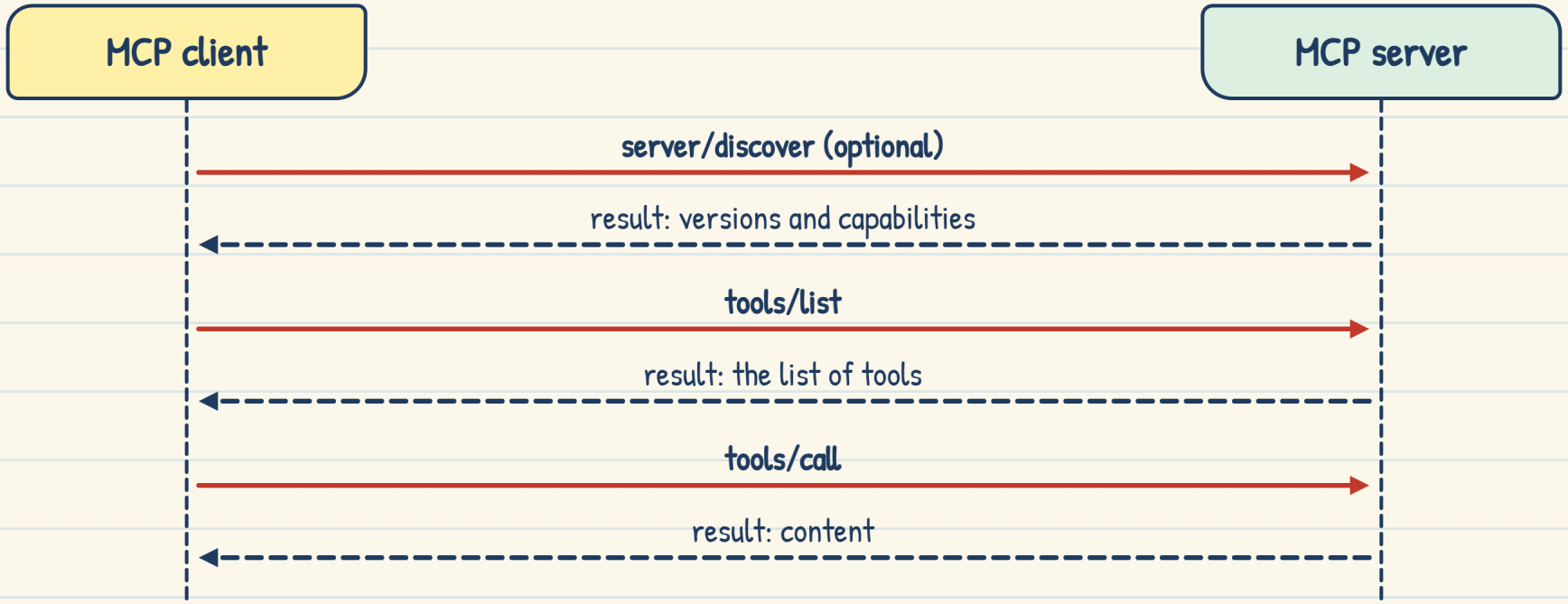
Templates. Reusable starting points: system prompts, few-shot examples.

Controlled by: the user picks them.

```
prompts/list
prompts/get
```

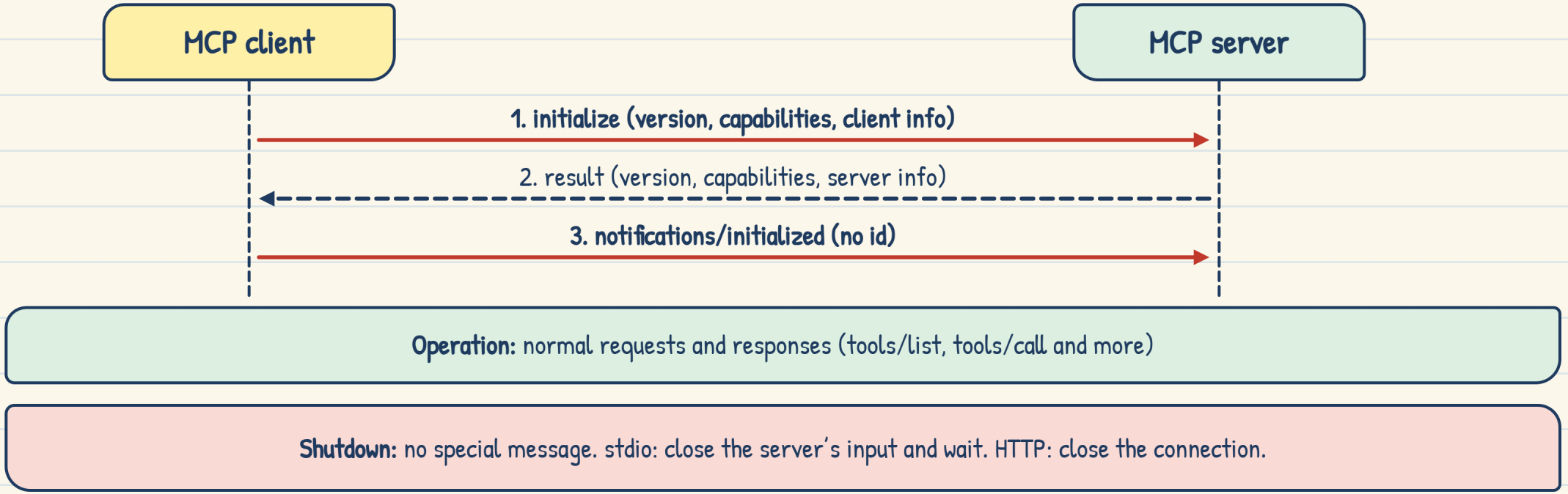
Example: a database server offers a tool to run queries, a resource with the table schema, and a prompt with example questions. Clients also offer **elicitation**, which lets a server ask the user for more input.

Connection flow today (2026-07-28)



No initialize step. Every request carries its own version and capabilities in _meta.

Older lifecycle: the handshake



Used by spec 2025-11-25 and earlier. Many tutorials and servers still use it.

JSON-RPC 2.0: the language of MCP

Request

Asks the server to do a job.

Has **jsonrpc, id, method, params**.

Needs an answer.

Response

Answers one request.

Has the **same id** as the request.

Holds a **result** or an **error**.

Notification

A one-way message.

Has **no id**.

No answer is sent back.

id = tracking number. Send id 1, and the answer comes back with id 1. Many requests can share one connection without confusion.

MCP rules: every message is plain JSON text (UTF-8). The id is a string or a number, never null, and it is not reused while a request is still waiting.

Anatomy of a request

tools/call request

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "weather_current",
    "arguments": {
      "location": "San Francisco"
    },
    "_meta": { ... }
  }
}
```

jsonrpc: the version. Always "2.0".

id: the tracking number. The response repeats it.

method: the job to do, like a function name.

params: the inputs. For a tool call: **name** (which tool) and **arguments**.

_meta: the protocol version and client capabilities. Every request carries them in 2026-07-28.

Anatomy of a response

Success response

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "resultType": "complete",
    "content": [
      { "type": "text",
        "text": "68°F, cloudy" }
    ]
  }
}
```

Error response

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "error": {
    "code": -32601,
    "message": "Method not found"
  }
}
```

Success has **result**. Failure has **error**. The id always matches the request.

Notifications: one-way messages

Notification: tools changed

```
{
  "jsonrpc": "2.0",
  "method":
    "notifications/tools/list_changed"
}
```

No id field. That is how you spot it.

The receiver must **not** reply. There is nothing to answer.

Example: the server says "my tools changed". The client then calls **tools/list** again.

In 2026-07-28 these are opt-in. The client opens a **subscriptions/listen** request and picks the types it wants.

A long task can also send **notifications/progress** while it runs.

Errors: two different kinds

1. Protocol error

The request itself is wrong, or the server broke. Examples: unknown tool, bad request shape, server failure.

```
{ "id": 3,
  "error": {
    "code": -32602,
    "message": "Unknown tool" }
}
```

2. Tool execution error

The tool ran but failed. The model reads the message and can try again. Examples: API failure, wrong date format.

```
{ "id": 4,
  "result": {
    "content": [ ... ],
    "isError": true }
}
```

Remember: an **error** object means a protocol problem. **isError: true** inside a normal result means the tool itself failed.

Error codes to know

Code	Name	Meaning
-32700	Parse error	The server could not read the JSON.
-32600	Invalid Request	The message is not a valid JSON-RPC request.
-32601	Method not found	The method does not exist, for example tools/lst.
-32602	Invalid params	Bad inputs. Also an unknown tool or a missing resource.
-32603	Internal error	Something broke inside the server.
-32020	HeaderMismatch	HTTP headers and the JSON body do not match.
-32021	MissingRequiredClientCapability	The client did not declare a capability the server needs.
-32022	UnsupportedProtocolVersion	The server does not support the requested version.

The first five rows are standard JSON-RPC codes. The blue rows are MCP codes from spec 2026-07-28.

Resources: read data

Request: resources/read

```
{
  "jsonrpc": "2.0", "id": 2,
  "method": "resources/read",
  "params": {
    "uri": "file:///app/notes.md"
  }
}
```

Response

```
{
  "jsonrpc": "2.0", "id": 2,
  "result": {
    "resultType": "complete",
    "contents": [ {
      "uri": "file:///app/notes.md",
      "mimeType": "text/markdown",
      "text": "# Notes ..." } ]
  }
}
```

resources/list shows what is available. **resources/read** gets the content of one resource, found by its URI.

Content is **text** or **blob** (binary, base64). A missing resource gives error -32602.

Transports: how messages travel

Question	stdio	Streamable HTTP
Best for	Local servers on your own machine	Remote servers on the internet
Who starts the server?	The client launches it as a subprocess	It runs on its own. Many clients connect
How do messages move?	One JSON message per line through stdin and stdout	Each message is an HTTP POST to one endpoint
How is sign-in done?	Credentials come from the environment	HTTP authorization. OAuth is recommended
Status	Current	Current. The old HTTP+SSE transport is deprecated

Local = stdio. Remote = Streamable HTTP. Old HTTP+SSE = do not use.

stdio transport (local)

MCP client

Inside the host app. It starts the server program.



stdin: client to server, one JSON line

stdout: server to client, one JSON line

stderr: logs only. Not part of the protocol.

MCP server

A subprocess on the same machine. Reads stdin, writes stdout.

1. One message per line. No line breaks inside a message.
2. Write only valid MCP messages to stdout. Print your logs to stderr.
3. To stop: the client closes the server's input, waits, and force-stops only if needed.

Streamable HTTP (remote)

One HTTP request (tools/call)

```
POST /mcp HTTP/1.1
Content-Type: application/json
Accept: application/json,
  text/event-stream
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: get_weather

{ ...the JSON-RPC request... }
```

One endpoint, for example **https://example.com/mcp**.

Every message is a new HTTP POST. A notification gets **202 Accepted**.

The reply is one JSON object, or an SSE stream (progress first, then the final result).

The Mcp-Method and Mcp-Name headers let gateways route requests without reading the body.

No session id and no GET stream in 2026-07-28. Both existed before.

SSE = Server-Sent Events: a server streams messages over one HTTP response.

Local server vs remote server

Local server (stdio)

- Runs on your own computer
- The AI app starts it for you
- Usually serves one client
- Has your user permissions
- Example: the File System server

Remote server (Streamable HTTP)

- Runs on the internet, hosted by someone
- You connect with a URL
- Usually serves many clients
- You sign in (OAuth is recommended)
- Example: the Sentry MCP server

"Local" and "remote" only say where the server runs. The MCP messages are the same.

Connect a local server (Claude Desktop)

1. Install Claude Desktop and Node.js. Check with node --version.
2. Claude menu, then Settings, then Developer, then Edit Config.
3. Add your server under "mcpServers" (the JSON on the right).
4. Save the file. Quit Claude Desktop fully, then start it again.
5. Open the connectors menu. Check that your server and its tools are listed. Approve each action when Claude asks.

claude_desktop_config.json (Windows)

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C:\\Users\\username\\Desktop"
      ]
    }
  }
}
```

Use full (absolute) paths. On Windows, write each backslash twice. Replace "username" with your own.

Local server: files and fixes

Where is everything?

Config file, Windows

```
%APPDATA%\Claude\claude_desktop_config.json
```

Config file, macOS

```
~/Library/Application
Support/Claude/claude_desktop_config.json
```

Logs, Windows and macOS

```
%APPDATA%\Claude\logs
~/Library/Logs/Claude
```

mcp.log = connection messages. **mcp-server-NAME.log** = what the server wrote to stderr.

Server not showing?

1. Restart Claude Desktop completely.
2. Check the JSON: commas, quotes, brackets.
3. Use absolute paths, not relative ones.
4. Read mcp-server-NAME.log for the error.
5. Run the same command by hand in a terminal and read its error.
6. Windows error with ${APPDATA}: add APPDATA to the server's "env" setting.

Connect a remote server (custom connector)

1. Open Claude (desktop app or browser). Go to Settings, then Connectors.
2. Click Add, then "Add custom connector".
3. Paste the server URL from its developer. Use the full https:// address with the path.
4. Click Add.
5. Sign in when asked (OAuth or another login).
6. Open the connector's settings. Turn tools on or off.
7. Use it from the "Add files, connectors, and more" menu in a chat.

Example server URL

```
https://example-server.modelcontextprotocol.io/mcp
```

What happens next: Claude sends MCP messages to that URL with HTTP POST. The server answers with JSON or a stream.

Be careful: connect only to servers you trust. Read the permissions you approve at sign-in.

Connect with Claude Code (commands)

Remote server (Streamable HTTP). The old "sse" option is deprecated.

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

Local server (stdio). The server command comes after the double dash.

```
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio airtable -- npx -y airtable-mcp-server
```

--scope local (default): private to you, this project.

--scope project: shared with your team in .mcp.json.

--scope user: available in all your projects.

Manage: claude mcp list, claude mcp get NAME, claude mcp remove NAME. Inside Claude Code, type **/mcp** to check status and sign in to OAuth servers.

Build your own server with FastMCP

server.py (Python)

```python
from fastmcp import FastMCP

mcp = FastMCP("MyServer")

@mcp.tool
def hello(name: str) -> str:
  return f"Hello, {name}!"

if __name__ == "__main__":
  mcp.run() # stdio
  # remote: mcp.run(transport="http", port=8000)
```

1. Write. A Python function becomes a tool. Type hints and the docstring describe it.

2. Run. mcp.run() uses stdio. With transport="http" the endpoint is http://localhost:8000/mcp.

3. Test. Open MCP Inspector (npx @modelcontextprotocol/inspector). Try tools/list, then tools/call.

4. Connect. Add it to Claude Desktop or Claude Code with the steps you just learned.

About MCP connectors

What is a connector?

In Claude, a connector is how Claude reaches an MCP server. You can connect many servers at the same time.

Remote connectors

Add them with a URL: Settings, Connectors, Add custom connector. They link Claude to remote MCP servers.

Local servers

Add them in the config file. After a restart they show in the connectors list too.

You stay in control

Turn single tools on or off. Approve actions. Remove connectors you no longer use.

Connector = the link inside Claude. MCP server = the program behind it.

What changed in spec 2026-07-28

Topic	Older (2025-11-25 and before)	Now (2026-07-28)
Start	initialize, then initialized	No handshake. Every request carries _meta. server/discover is optional.
Sessions	Mcp-Session-Id header	No protocol session. Servers are stateless. Tools can return a handle (an id) for state.
Server asks client	Server sends its own request on a stream	Server replies input_required. The client retries with the answers.
Change alerts	A GET stream	A subscriptions/listen stream. The client opts in.
HTTP headers	MCP-Protocol-Version	Also Mcp-Method and Mcp-Name, so gateways can route.

Many tutorials and older servers still use the left column. Modern clients can fall back to it.

Deprecated: do not start with these

Item	Status	What to do
HTTP+SSE transport	Deprecated	Use Streamable HTTP
Roots	Deprecated	New projects should not adopt it
Sampling	Deprecated	Call an LLM provider API directly
Logging feature	Deprecated	stdio: log to stderr. Or use OpenTelemetry
Dynamic Client Registration	Deprecated	Use Client ID Metadata Documents
Tasks	Moved out of core	Now an extension

Deprecated does not mean removed. The spec keeps deprecated features for at least twelve months. Still current: tools, resources, prompts, elicitation and extensions such as MCP Apps.

Safety checklist

- ✓ Connect only to servers you trust.
- ✓ Share the smallest set of folders and data.
- ✓ Read what a tool will do before you approve it.
- ✓ Keep secrets out of chat and URLs. Pass keys as environment variables.
- ✓ Remote servers: sign in with OAuth and review the permissions.
- ✓ Servers you run over HTTP locally: bind to 127.0.0.1 and check the Origin header.
- ✓ Treat tool descriptions from unknown servers as untrusted.
- ✓ Remove connectors you no longer use.

A person should always be able to say no to a tool call.

One-page cheat sheet

Roles

Host: the AI app.

Client: part of the host, one per server.

Server: gives tools, data, prompts.

Primitives

Tools: actions.

Resources: data to read.

Prompts: templates.

Messages

Request: has an id.

Response: same id, result or error.

Notification: no id.

Methods

server/discover, tools/list, tools/call, resources/list, resources/read, prompts/list, prompts/get, subscriptions/listen

Transports

Local: **stdio**.

Remote: **Streamable HTTP**.

HTTP+SSE: deprecated.

Errors

-32700 parse, -32600 invalid request, -32601 method not found, -32602 invalid params, -32603 internal. isError: true = the tool failed.

Remember this

Agentic RAG builds the whole house for the AI. MCP gives the AI a key to your house.

Your project is not replaced. It gets a new way to be used.

MCP is an open standard that connects AI apps to tools and data.

Host, client, server: one client for each server.

Messages are JSON-RPC: request, response, notification.

Local uses stdio. Remote uses Streamable HTTP.

Since 2026-07-28: no handshake, no sessions.

Next class: build a local server with FastMCP, then run tools/list and tools/call.